

Student Performance Analyzer

February 17, 2026

```
[1]: import numpy as np
import pandas as pd

# ----- Exception Handling -----
def safe_input(prompt, cast_type=str):
    while True:
        try:
            value = cast_type(input(prompt))
            return value
        except ValueError:
            print(f"Invalid input! Please enter a valid {cast_type.__name__}.")

# ----- OOP Classes -----
class Student:
    def __init__(self, name, roll_no, age, class_name, marks):
        self.name = name
        self.roll_no = roll_no
        self.age = age
        self.class_name = class_name
        self.marks = marks # dictionary of subject: marks

    def display_details(self):
        print(f"\nName: {self.name}, Roll No: {self.roll_no}, "
              f"Age: {self.age}, Class: {self.class_name}")
        print("Marks:", self.marks)

    def calculate_average(self):
        return np.mean(list(self.marks.values()))

    def calculate_grade(self):
        avg = self.calculate_average()
        if avg >= 90:
            return "A"
        elif avg >= 75:
            return "B"
        elif avg >= 60:
            return "C"
```

```

        elif avg >= 40:
            return "D"
        else:
            return "F"

class Classroom:
    def __init__(self):
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def remove_student(self, roll_no):
        self.students = [s for s in self.students if s.roll_no != roll_no]

    def overall_performance(self):
        if not self.students:
            return None
        all_marks = [s.calculate_average() for s in self.students]
        return {
            "Mean": np.mean(all_marks),
            "Median": np.median(all_marks),
            "Std Dev": np.std(all_marks)
        }

    def to_dataframe(self):
        data = []
        for s in self.students:
            row = {
                "Name": s.name,
                "Roll No": s.roll_no,
                "Age": s.age,
                "Class": s.class_name,
                "Average Marks": s.calculate_average(),
                "Grade": s.calculate_grade()
            }
            row.update(s.marks) # add subjects
            data.append(row)
        return pd.DataFrame(data)

# ----- Main Program -----
def main():
    classroom = Classroom()

    print("----- Student Performance Analyzer -----")

```

```

n = safe_input("Enter number of students: ", int)

for _ in range(n):
    print("\nEnter details of student:")
    name = safe_input("Name: ", str)
    roll_no = safe_input("Roll No: ", int)
    age = safe_input("Age: ", int)
    class_name = safe_input("Class: ", str)

    marks = {}
    subjects = ["Math", "Science", "English"]
    for sub in subjects:
        marks[sub] = safe_input(f"Enter marks in {sub}: ", int)

    student = Student(name, roll_no, age, class_name, marks)
    classroom.add_student(student)

# Display all students
print("\n----- Student Details -----")
for s in classroom.students:
    s.display_details()
    print("Average:", s.calculate_average(), "Grade:", s.calculate_grade())

# NumPy Analysis
print("\n----- NumPy Analysis -----")
all_marks = np.array([list(s.marks.values()) for s in classroom.students])
print("Average per subject:", np.mean(all_marks, axis=0))
print("Highest per subject:", np.max(all_marks, axis=0))
print("Lowest per subject:", np.min(all_marks, axis=0))
print("Overall Performance:", classroom.overall_performance())

# Pandas Analysis
print("\n----- Pandas Analysis -----")
df = classroom.to_dataframe()
print("\nDataFrame:\n", df)
print("\nTop 5 Students by Avg Marks:\n", df.nlargest(5, "Average Marks"))
print("\nBottom 5 Students by Avg Marks:\n", df.nsmallest(5, "Average_
↵Marks"))
print("\nGroup by Class (Avg Marks):\n",
      df.groupby("Class")["Average Marks"].mean())

# Save to file
try:
    df.to_csv("student_report.csv", index=False)
    print("\nReport saved as 'student_report.csv'")
except Exception as e:
    print("Error saving file:", e)

```

```
# Run Program
if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print("Unexpected error:", e)
```

----- Student Performance Analyzer -----

Enter number of students: 2

Enter details of student:

Name: Swapnil

Roll No: 100

Age: 22

Class: BSC Computer Science

Enter marks in Math: 91

Enter marks in Science: 90

Enter marks in English: 93

Enter details of student:

Name: Shreyas

Roll No: 101

Age: 22

Class: BSC Computer Science

Enter marks in Math: 89

Enter marks in Science: 91

Enter marks in English: 90

----- Student Details -----

Name: Swapnil , Roll No: 100, Age: 22, Class: BSC Computer Science

Marks: {'Math': 91, 'Science': 90, 'English': 93}

Average: 91.33333333333333 Grade: A

Name: Shreyas, Roll No: 101, Age: 22, Class: BSC Computer Science

Marks: {'Math': 89, 'Science': 91, 'English': 90}

Average: 90.0 Grade: A

----- NumPy Analysis -----

Average per subject: [90. 90.5 91.5]

Highest per subject: [91 91 93]

Lowest per subject: [89 90 90]

Overall Performance: {'Mean': 90.66666666666666, 'Median': 90.66666666666666, 'Std Dev': 0.6666666666666643}

----- Pandas Analysis -----

DataFrame:

	Name	Roll No	Age	Class	Average Marks	Grade	Math	\
0	Swapnil	100	22	BSC Computer Science	91.333333	A	91	
1	Shreyas	101	22	BSC Computer Science	90.000000	A	89	

	Science	English
0	90	93
1	91	90

Top 5 Students by Avg Marks:

	Name	Roll No	Age	Class	Average Marks	Grade	Math	\
0	Swapnil	100	22	BSC Computer Science	91.333333	A	91	
1	Shreyas	101	22	BSC Computer Science	90.000000	A	89	

	Science	English
0	90	93
1	91	90

Bottom 5 Students by Avg Marks:

	Name	Roll No	Age	Class	Average Marks	Grade	Math	\
1	Shreyas	101	22	BSC Computer Science	90.000000	A	89	
0	Swapnil	100	22	BSC Computer Science	91.333333	A	91	

	Science	English
1	91	90
0	90	93

Group by Class (Avg Marks):

```
Class
BSC Computer Science    90.666667
Name: Average Marks, dtype: float64
```

Report saved as 'student_report.csv'

[]: